02-VM-네이티브-컴파일

개요

Monad의 VM은 EVM 바이트코드를 네이티브 x86 머신 코드로 JIT(Just-In-Time) 컴파일하여 극한의 실행 성능을 달성합니다. 이 문서는 VM의 인터프리터 구조, JIT 컴파일 파이프라인, 비동기 컴파일 전략, 그리고 코드 캐싱 메커니즘을 상세히 분석합니다.

핵심 최적화 기법

1. **하이브리드 실행 모델**: 인터프리터와 JIT 컴파일러를 동적으로 전환
2. **비동기 컴파일**: 백그라운드 스레드에서 컴파일하여 실행 지연 최소화
3. **가중치 기반 LRU 캐싱**: 컴파일된 코드를 효율적으로 캐싱
4. **asmjit 기반 코드 생성**: 고성능 x86 네이티브 코드 생성
5. **LLVM 백엔드 지원**: 선택적으로 LLVM을 통한 고급 최적화

관련 파일

인터프리터:

- `category/vm/interpreter/execute.cpp` : 인터프리터 실행 루프
- `category/vm/interpreter/intercode.cpp` : 중간 표현(Intercode) 생성
- `category/vm/interpreter/instruction_table.hpp` : 명령어 테이블 및 구현
- `category/vm/interpreter/stack.hpp` : 스택 관리 및 검증

컴파일러:

- `category/vm/compiler.cpp` : 컴파일러 메인 로직 및 비동기 컴파일

- `category/vm/compiler.hpp` : 컴파일러 인터페이스
- `category/vm/compiler/ir/x86.hpp` : x86 코드 생성
- `category/vm/varcode_cache.cpp` : 코드 캐시 구현

VM 메인:

- `category/vm/vm.cpp` : VM 실행 진입점 및 실행 전략

LLVM 백엔드:

- `category/vm/llvm/llvm.cpp` : LLVM JIT 통합

1. EVM 인터프리터 구조

1.1 실행 루프 (Execute Loop)

인터프리터는 EVM 바이트코드를 직접 해석하여 실행하는 폴백 메커니즘입니다. 컴파일된 코드가 없거나 컴파일이 진행 중일 때 사용됩니다.

실행 진입점

```
// category/vm/interpreter/execute.cpp
template <Traits traits>
void  execute(
runtime::Context &ctx,
Intercode const  &analysis,
std::uint8_t  *stack_ptr)
{
monad_vm_interpreter_trampoline(
static_cast<void *>(&ctx.exit_stack_ptr),
```

```
&ctx,
&analysis,
reinterpret_cast<runtime::uint256_t *>(stack_ptr),
reinterpret_cast<void *>(core_loop<traits>));
}
```

실행 흐름:

1. `monad_vm_interpreter_trampoline` (어셈블리 함수)가 스택을 설정
2. `core_loop<traits>` 로 점프하여 실제 실행 시작
3. 명령어 테이블을 통해 각 옵코드 실행

핵심 실행 루프

```
template <Traits traits>
void  core_loop(
void  *, runtime::Context *ctx, Intercode const  *analysis,
runtime::uint256_t *stack_ptr, void  *)
{
auto *const stack_top = stack_ptr - 1;
auto  const *const stack_bottom = stack_top;
auto  const *const instr_ptr = analysis->code();
auto  const gas_remaining = ctx->gas_remaining;

// 첫 번째 명령어 실행
```

```
instruction_table<traits>[*instr_ptr](
*ctx,
*analysis,
stack_bottom,
stack_top,
gas_remaining,
instr_ptr);
}
```

핵심 특징:

- **Tail Call 최적화**: `MONAD_VM_MUST_TAIL` 속성으로 스택 오버플로우 방지
- **직접 스레드 디스패치**: 함수 포인터 테이블을 통한 빠른 명령어 디스패치
- **스택 포인터 추적**: 각 명령어가 스택 변화량(delta)을 계산하여 다음 명령어로 전달

1.2 중간 표현 (Intercode)

Intercode는 EVM 바이트코드를 파싱하여 생성되는 중간 표현으로, 인터프리터와 컴파일러 모두에서 사용됩니다.

Intercode 생성

```
// category/vm/interpreter/intercode.cpp
Intercode::Intercode(std::span<std::uint8_t  const> const code)
: padded_code_(pad(code))
, code_size_(code_size_t::unsafe_from(static_cast<uint32_t>(code.size())))
, jumpdest_map_(find_jumpdests(code))
```

```
{

}
```

구성 요소:

1. **padded_code_**: 패딩된 바이트코드 (시작/끝에 0 추가)
2. **code_size_**: 원본 코드 크기
3. **jumpdest_map_**: JUMPDEST 명령어 위치 비트맵

JUMPDEST 맵 생성

```
auto Intercode::find_jumpdests(std::span<std::uint8_t  const> const  code)
-> JumpdestMap
{
auto jumpdests = JumpdestMap(code.size(), false);

for (auto i = 0u; i < code.size(); ++i) {
auto  const op = code[i];

if (op == EvmOpCode::JUMPDEST) {
jumpdests[i] = true;
}

// PUSH 명령어의 즉시값은 건너뛰기
if (is_push_opcode(op)) {
i += get_push_opcode_index(op);
```

```
}
}

return jumpdests;
}
```

최적화 효과:

- 점프 대상 검증을 O(1)로 수행
- 컴파일 시 기본 블록 분석에 활용
- 잘못된 점프 시도를 빠르게 감지

1.3 명령어 테이블 (Instruction Table)

명령어 테이블은 256개의 EVM 옵코드를 함수 포인터 배열로 매핑합니다.

테이블 생성

```
// category/vm/interpreter/instruction_table.hpp
template <Traits traits>
consteval InstrTable make_instruction_table()
{
constexpr  auto since = [](evmc_revision first, InstrEval impl) {
return (traits::evm_rev() >= first) ? impl : invalid;
};
```

```
return {
stop, // 0x00
add<traits>, // 0x01
mul<traits>, // 0x02
sub<traits>, // 0x03
// ... 256개 명령어
since(EVMC_CONSTANTINOPLE, shl<traits>), // 0x1B
since(EVMC_CANCUN, tload<traits>), // 0x5C
selfdestruct<traits>, // 0xFF
};
}
```

```
template <Traits traits>
constexpr InstrTable instruction_table = make_instruction_table<traits>();
```

특징:

- **컴파일 타임 생성**: `consteval` 로 컴파일 시점에 테이블 생성
- **버전별 명령어**: EVM 버전에 따라 명령어 활성화/비활성화
- **타입 안전성**: 템플릿으로 각 EVM 버전별 독립적인 테이블

명령어 구현 예시

```
// ADD 명령어 (간단한 연산)
template <Traits traits>
MONAD_VM_INSTRUCTION_CALL void
```

```
add(runtime::Context &ctx, Intercode const &analysis,
runtime::uint256_t const *stack_bottom, runtime::uint256_t *stack_top,
std::int64_t gas_remaining, std::uint8_t  const *instr_ptr)
{
// 가스 및 스택 검증
check_requirements<ADD, traits>(
ctx, analysis, stack_bottom, stack_top, gas_remaining);
// 스택 상위 2개 값 가져오기
auto &&[a, b] = top_two(stack_top);
b = a + b; // 덧셈 수행


// 다음 명령어로 tail call
MONAD_VM_NEXT(ADD);
}


// MUL 명령어 (런타임 함수 호출)
template <Traits traits>
MONAD_VM_INSTRUCTION_CALL void
mul(runtime::Context &ctx, Intercode const &analysis,
runtime::uint256_t const *stack_bottom, runtime::uint256_t *stack_top,
std::int64_t gas_remaining, std::uint8_t  const *instr_ptr)
{
```

```
// 검증 후 런타임 함수 호출
checked_runtime_call<MUL, traits>(
monad_vm_runtime_mul, // 어셈블리 최적화된 곱셈 함수
ctx, analysis, stack_bottom, stack_top, gas_remaining, instr_ptr);


MONAD_VM_NEXT(MUL);
}
```

MONAD_VM_NEXT 매크로:

```
#define  MONAD_VM_NEXT(OP) \
do { \
static  constexpr  auto delta = \
compiler::opcode_table<traits>[(OP)].stack_increase - \
compiler::opcode_table<traits>[(OP)].min_stack; \
\
++instr_ptr; // 다음 명령어로 이동 \
MONAD_VM_MUST_TAIL return instruction_table<traits>[*instr_ptr]( \
ctx, analysis, stack_bottom, \
stack_top + delta, // 스택 포인터 조정 \
gas_remaining, instr_ptr); \
} while (false);
```

1.4 스택 관리 (Stack Management)

스택 관리는 EVM의 1024 깊이 제한과 가스 소비를 검증합니다.

요구사항 검증

```
// category/vm/interpreter/stack.hpp
template <std::uint8_t Instr, Traits traits>
[[gnu::always_inline]] inline void check_requirements(
runtime::Context &ctx, Intercode const &,
runtime::uint256_t const *stack_bottom, runtime::uint256_t *stack_top,
std::int64_t &gas_remaining)
{
static constexpr auto info = compiler::opcode_table<traits>[Instr];

// 가스 검증
if constexpr (info.min_gas > 0) {
gas_remaining -= info.min_gas;
if (MONAD_VM_UNLIKELY(gas_remaining < 0)) {
ctx.exit(OutOfGas);
}
}

auto const stack_size = stack_top - stack_bottom;

// 스택 언더플로우 검증
```

```
if constexpr (info.min_stack > 0) {
if (MONAD_VM_UNLIKELY(stack_size < info.min_stack)) {
ctx.exit(Error);
}
}

// 스택 오버플로우 검증
if constexpr (info.stack_increase > 0) {
static constexpr auto delta = info.stack_increase - info.min_stack;
static constexpr auto max_safe_size = 1024 - delta;

if constexpr (max_safe_size < 1024) {
if (MONAD_VM_UNLIKELY(stack_size > max_safe_size)) {
ctx.exit(Error);
}
}
}
}
```

최적화 기법:

- **컴파일 타임 검증**: `if constexpr` 로 불필요한 검증 제거
- **분기 예측 힌트**: `MONAD_VM_UNLIKELY` 매크로로 에러 경로 최적화
- **정적 정보 활용**: 각 명령어의 스택 요구사항을 컴파일 타임에 계산

스택 조작 헬퍼 함수

```
// 스택에 값 푸시
[[gnu::always_inline]] inline  void
push(runtime::uint256_t *stack_top, runtime::uint256_t const &x)
{
*(stack_top + 1) = x;
}

// 스택에서 값 팝
[[gnu::always_inline]] inline runtime::uint256_t &
pop(runtime::uint256_t *&stack_top)
{
return *stack_top--;
}

// 스택 상위 2개 값 참조
[[gnu::always_inline]] inline  auto
top_two(runtime::uint256_t *const stack_top)
{
auto  const &a = *stack_top;
return std::tie(a, *(stack_top - 1));
}
```

2. JIT 컴파일 파이프라인

2.1 컴파일러 아키텍처

Monad의 JIT 컴파일러는 asmjit 라이브러리를 사용하여 EVM 바이트코드를 x86 네이티브 코드로 변환합니다.

컴파일러 클래스 구조

```
// category/vm/compiler.hpp

class Compiler

{

asmjit::JitRuntime asmjit_rt_; // asmjit JIT 런타임

VarcodeCache varcode_cache_; // 컴파일된 코드 캐시

CompileJobMap compile_job_map_; // 컴파일 작업 맵

CompileJobQueue compile_job_queue_; // 컴파일 작업 큐

std::thread compiler_thread_; // 백그라운드 컴파일 스레드

std::atomic_flag stop_flag_; // 종료 플래그

size_t compile_job_soft_limit_; // 작업 큐 소프트 리밋

bool enable_async_compilation_; // 비동기 컴파일 활성화



public:

// 동기 컴파일

template <Traits traits>

SharedNativecode compile(

SharedIntercode const  &,
```

```
CompilerConfig const  & = {});


// 캐시된 컴파일
template <Traits traits>
SharedNativecode cached_compile(
evmc::bytes32 const  &code_hash,
SharedIntercode const  &,
CompilerConfig const  & = {});


// 비동기 컴파일
template <Traits traits>
bool  async_compile(
evmc::bytes32 const  &code_hash,
SharedIntercode const  &,
CompilerConfig const  & = {});
};
```

2.2 컴파일 프로세스

동기 컴파일

```
// category/vm/compiler.cpp
template <Traits traits>
SharedNativecode Compiler::compile(
```

```
SharedIntercode const  &icode, CompilerConfig const  &config)
{
return compiler::native::compile<traits>(
asmjit_rt_,
icode->code(),
icode->code_size(),
config);
}
```

컴파일 파이프라인:

```
EVM Bytecode
↓
Intercode (중간 표현)
↓
Basic Block 분석
↓
asmjit IR 생성
↓
x86 Machine Code
↓
Nativecode (실행 가능)
```

캐시된 컴파일

```
template <Traits traits>
SharedNativecode Compiler::cached_compile(
evmc::bytes32 const  &code_hash,
SharedIntercode const  &icode,
CompilerConfig const  &config)
{
// 캐시 조회
if (auto vcode = varcode_cache_.get(code_hash)) {
auto  const &ncode = (*vcode)->nativecode();
if (ncode != nullptr && ncode->chain_id() == traits::id()) {
return ncode; // 캐시 히트
}
}
// 캐시 미스: 컴파일 수행
auto  const start = std::chrono::steady_clock::now();
auto ncode = compile<traits>(icode, config);
auto  const end = std::chrono::steady_clock::now();
// 캐시에 저장
varcode_cache_.set(code_hash, icode, ncode);
stats_.event_new_compiled_code_cached(icode, ncode, start, end);
return ncode;
}
```

2.3 asmjit 통합

asmjit은 런타임에 x86/x64 어셈블리 코드를 생성하는 라이브러리입니다.

JIT 런타임 설정

```
static constexpr asmjit::JitAllocator::CreateParams
asmjit_create_params_{
.options = asmjit::JitAllocatorOptions::kUseDualMapping,
};

asmjit::JitRuntime asmjit_rt_{&asmjit_create_params_};
```

Dual Mapping 최적화:

- 코드 메모리를 읽기/쓰기와 실행 권한으로 이중 매핑
- W^X (Write XOR Execute) 보안 정책 준수
- 코드 생성과 실행을 효율적으로 분리

코드 생성 인터페이스

```
// category/vm/compiler/ir/x86.hpp
template <Traits traits>
std::shared_ptr<Nativecode> compile(
asmjit::JitRuntime &rt,
std::uint8_t const *contract_code,
interpreter::code_size_t contract_code_size,
```

```
CompilerConfig const  & = {});
```

코드 크기 제한:

```
constexpr native_code_size_t max_code_size(
interpreter::code_size_t offset,
interpreter::code_size_t bytecode_size) noexcept
{
// 네이티브 코드 크기 = offset + (bytecode_size << 5)
// 바이트코드 1바이트당 최대 32바이트의 네이티브 코드 생성
return offset + shl<5>(bytecode_size);
}
```

2.4 타입 추론 및 최적화

컴파일러는 스택 값의 타입을 추론하여 최적화된 코드를 생성합니다.

최적화 기법:

1. **상수 전파**: 컴파일 타임에 알려진 값을 즉시 계산
2. **레지스터 할당**: 자주 사용되는 값을 레지스터에 유지
3. **불필요한 검증 제거**: 정적 분석으로 안전성이 보장된 경우 검증 생략
4. **인라인 확장**: 작은 런타임 함수를 인라인으로 전개

3. 비동기 컴파일 및 캐싱 전략

3.1 비동기 컴파일 메커니즘

비동기 컴파일은 백그라운드 스레드에서 수행되어 실행 지연을 최소화합니다.

비동기 컴파일 요청

```
template <Traits traits>
bool Compiler::async_compile(
evmc::bytes32 const  &code_hash,
SharedIntercode const  &icode,
CompilerConfig const  &config)
{
// 작업 큐 크기 제한 확인
if (compile_job_map_.size() >= compile_job_soft_limit_) {
return false; // 큐가 가득 참
}
// 컴파일 람다 생성
auto  const cached_compile_lambda = [this](auto  &&...args) {
return  this->cached_compile<traits>(
std::forward<decltype(args)>(args)...);
};


// 작업 맵에 삽입
if (!compile_job_map_.insert(
{code_hash, {cached_compile_lambda, traits::id(), icode, config}})) {
return false; // 이미 제출된 작업
```

```
}

// 큐에 추가 및 스레드 깨우기

compile_job_queue_.push(code_hash);

compile_job_cv_.notify_all();

return true;

}
```

백그라운드 컴파일 루프

```
void Compiler::compile_loop()

{

while (!stop_flag_.test(std::memory_order_acquire)) {

// 최대 1ms 대기 (일반적인 컨트랙트 컴파일 시간)

compile_job_cv_.wait_for(

compile_job_lock_,

std::chrono::milliseconds{1});

dispense_compile_jobs();

}

}


void Compiler::dispense_compile_jobs()

{

evmc::bytes32 code_hash;

while (compile_job_queue_.try_pop(code_hash) &&
```

```
!stop_flag_.test(std::memory_order_acquire)) {
CompileJobAccessor acc;
bool const find_ok = compile_job_map_.find(acc, code_hash);
auto const &[compile_fn, chain_id, icode, config] = acc->second;


if (MONAD_VM_LIKELY(enable_async_compilation_)) {
// 컴파일 수행 (캐시 확인 포함)
compile_fn(code_hash, icode, config);
}


// 작업 맵에서 제거
bool const erase_ok = compile_job_map_.erase(acc);
}
}
```

동시성 제어:

- `tbb::concurrent_hash_map` : 스레드 안전한 작업 맵
- `tbb::concurrent_queue` : 락 프리 작업 큐
- `std::condition_variable` : 스레드 간 동기화

3.2 Varcode 캐시 구현

VarcodeCache는 가중치 기반 LRU 캐시로 컴파일된 코드를 관리합니다.

캐시 구조

```
// category/vm/varcode_cache.hpp
class VarcodeCache
{
static  constexpr std::uint32_t default_max_cache_kb =
std::uint32_t{1} << 22; // 4GB

static  constexpr std::uint32_t default_warm_cache_kb =
(3 * default_max_cache_kb) / 4; // ~75%

using WeightCache = utils::LruWeightCache<
evmc::bytes32, SharedVarcode, utils::Hash32Compare>;

WeightCache weight_cache_;
std::uint32_t warm_cache_kb_;
};
```

가중치 계산

```
std::uint32_t VarcodeCache::code_size_to_cache_weight(
std::uint32_t  code_size)
{
// 코드 크기(kB) + 3kB 오버헤드
return (code_size >> 10) + 3;
```

```
}
```

가중치 기반 LRU:

- 코드 크기에 비례한 가중치 할당
- 큰 컨트랙트는 더 빨리 축출
- 작은 컨트랙트는 더 오래 유지

캐시 조회 및 삽입

```
// 캐시 조회
std::optional<SharedVarcode> VarcodeCache::get(
evmc::bytes32 const  &code_hash)
{
WeightCache::ConstAccessor acc;
if (!weight_cache_.find(acc, code_hash)) {
return std::nullopt;
}
return  acc->second.value_;
}

// 캐시 삽입 (Intercode + Nativecode)
void VarcodeCache::set(
evmc::bytes32 const  &code_hash,
SharedIntercode const  &icode,
```

```
SharedNativecode const  &ncode)
{
auto weight = code_size_to_cache_weight(
*(icode->code_size() + ncode->code_size_estimate()));
auto vcode = std::make_shared<Varcode>(icode, ncode);
weight_cache_.insert(code_hash, vcode, weight);
}


// 캐시 삽입 시도 (Intercode만)
SharedVarcode VarcodeCache::try_set(
evmc::bytes32 const  &code_hash,
SharedIntercode const  &icode)
{
auto weight = code_size_to_cache_weight(*icode->code_size());
auto vcode = std::make_shared<Varcode>(icode);
(void)weight_cache_.try_insert(code_hash, vcode, weight);
return vcode;
}
```

Warm 캐시 전략

```
bool  is_warm()
{
return  weight_cache_.approx_weight() >= warm_cache_kb_;
```

```
}
```

Warm 캐시 동작:

- 캐시가 75% 이상 채워지면 “warm” 상태
- Warm 상태가 아니면 즉시 비동기 컴파일 시작
- Warm 상태에서는 가스 임계값 도달 시 컴파일

3.3 TBB 동시성 라이브러리 활용

Intel TBB(Threading Building Blocks)를 사용하여 스레드 안전한 자료구조를 구현합니다.

사용된 TBB 컴포넌트:

1. `tbb::concurrent_hash_map` : 동시 접근 가능한 해시맵
2. `tbb::concurrent_queue` : 락 프리 큐

장점:

- 높은 동시성 성능
- 락 프리 알고리즘
- 확장 가능한 병렬 처리

4. VM 실행 전략

4.1 하이브리드 실행 모델

VM은 인터프리터와 컴파일된 코드를 동적으로 전환하여 최적의 성능을 달성합니다.

실행 진입점

```
// category/vm/vm.cpp
```

```
template <Traits traits>
evmc::Result VM::execute(
Host &host,
evmc_message const  *msg,
evmc::bytes32 const  &code_hash,
SharedVarcode const  &vcode)
{
auto  const &icode = vcode->intercode();
auto rt_ctx = runtime::Context::from(
memory_allocator_, host_itf, host_ctx, msg, icode->code_span());

// 런타임 컨텍스트 설치
auto *const prev_rt_ctx = host.set_runtime_context(&rt_ctx);

auto result = execute_impl<traits>(rt_ctx, code_hash, vcode);

// 이전 컨텍스트 복원
(void)host.set_runtime_context(prev_rt_ctx);

return result;
}
```

실행 전략 결정

```
template <Traits traits>
evmc::Result VM::execute_impl(
runtime::Context &rt_ctx,
evmc::bytes32 const  &code_hash,
SharedVarcode const  &vcode)
{
auto  const &icode = vcode->intercode();
auto  const &ncode = vcode->nativecode();
auto  const msg_gas = rt_ctx.gas_remaining;
if (MONAD_VM_LIKELY(ncode != nullptr)) {
// 네이티브 코드가 존재하는 경우
if (MONAD_VM_UNLIKELY(ncode->chain_id() != traits::id())) {
// 버전 불일치: 재컴파일 시작 후 인터프리터 실행
compiler_.async_compile<traits>(code_hash, icode, compiler_config_);
return  execute_intercode_impl<traits>(rt_ctx, icode);
}
auto  const entry = ncode->entrypoint();
if (MONAD_VM_UNLIKELY(entry == nullptr)) {
// 컴파일 실패: 인터프리터로 폴백
return  execute_intercode_impl<traits>(rt_ctx, icode);
}
// 네이티브 코드 실행
```

```
return execute_native_entrypoint_impl(rt_ctx, entry);
}
// 네이티브 코드가 없는 경우
if (!compiler_.is_varcode_cache_warm()) {
// 캐시가 warm이 아니면 즉시 컴파일 시작
compiler_.async_compile<traits>(code_hash, icode, compiler_config_);
return execute_intercode_impl<traits>(rt_ctx, icode);
}
// 인터프리터로 실행
auto result = execute_intercode_impl<traits>(rt_ctx, icode);
// 가스 사용량 기반 컴파일 트리거
auto const bound = compiler::native::max_code_size(
compiler_config_.max_code_size_offset, icode->code_size());
uint64_t const gas_used =
static_cast<uint64_t>(msg_gas - result.gas_left);
if (vcode->intercode_gas_used(gas_used) >= *bound) {
// 누적 가스가 임계값 초과: 비동기 컴파일 시작
compiler_.async_compile<traits>(code_hash, icode, compiler_config_);
}
return result;
}
```

실행 전략 결정 트리:

```
네이티브 코드 존재?
├─ Yes
│ ├─ 버전 일치?
│ │ ├─ Yes
│ │ │ ├─ 엔트리포인트 유효?
│ │ │ │ ├─ Yes → 네이티브 코드 실행
│ │ │ │ └─ No → 인터프리터 실행
│ │ └─ No → 재컴파일 시작 + 인터프리터 실행
│
└─ No
├─ 캐시 Warm?
│ ├─ Yes → 인터프리터 실행 + 가스 임계값 확인
│ └─ No → 즉시 컴파일 시작 + 인터프리터 실행
```

인터프리터 실행

```
template <Traits traits>
evmc::Result VM::execute_intercode_impl(
runtime::Context &rt_ctx,
SharedIntercode const  &icode)
{
stats_.event_execute_intercode();
```

```
auto const stack_ptr = stack_allocator_.allocate();
interpreter::execute<traits>(rt_ctx, *icode, stack_ptr.get());

return rt_ctx.copy_to_evmc_result();
}
```

네이티브 코드 실행

```
evmc::Result VM::execute_native_entrypoint_impl(
runtime::Context &rt_ctx,
compiler::native::entrypoint_t entry)
{
stats_.event_execute_native_entrypoint();

auto const stack_ptr = stack_allocator_.allocate();
entry(&rt_ctx, stack_ptr.get()); // 네이티브 함수 호출

return rt_ctx.copy_to_evmc_result();
}
```

4.2 가스 기반 컴파일 트리거

컴파일 여부는 누적 가스 사용량으로 결정됩니다.

임계값 계산:

```
auto  const bound = compiler::native::max_code_size(
compiler_config_.max_code_size_offset,
icode->code_size());


// bound = offset + (bytecode_size << 5)
// 예: 128kB 바이트코드 → 4M 가스 임계값
```

누적 가스 추적:

```
if (vcode->intercode_gas_used(gas_used) >= *bound) {
compiler_.async_compile<traits>(code_hash, icode, compiler_config_);
}
```

최적화 효과:

- 자주 실행되는 코드만 컴파일
- 일회성 코드는 인터프리터로 실행
- 컴파일 오버헤드 최소화

5. LLVM 백엔드 (선택적)

Monad는 asmjit 외에도 LLVM을 통한 고급 최적화를 지원합니다.

5.1 LLVM VM 구조

```
// category/vm/llvm/llvm.cpp
```

```
class VM
{
runtime::EvmStackAllocator stack_allocator_;
runtime::EvmMemoryAllocator memory_allocator_;
// EVM 버전별 LLVM 코드 캐시
std::vector<std::unordered_map<evmc::bytes32, std::shared_ptr<LLVMState>>>
cached_llvm_code_;


public:
VM(std::size_t max_stack_cache, std::size_t max_memory_cache);
// LLVM 코드 캐싱
std::shared_ptr<LLVMState> cache_llvm(
evmc_revision rev,
evmc::bytes32 const &code_hash,
uint8_t const *code,
size_t code_size);
// LLVM 실행
evmc::Result execute_llvm(
evmc_revision rev,
evmc::bytes32 const &code_hash,
evmc_host_interface const *host,
evmc_host_context *context,
evmc_message const *msg,
```

```
uint8_t const *code,
size_t code_size);
};
```

5.2 LLVM 컴파일 및 실행

```
std::shared_ptr<LLVMState> VM::cache_llvm(
evmc_revision rev,
evmc::bytes32 const &code_hash,
uint8_t const *code,
size_t code_size)
{
// 캐시 조회
auto item = cached_llvm_code_[rev].find(code_hash);
if (item != cached_llvm_code_[rev].end()) {
return item->second;
}

// LLVM 컴파일
auto ptr = monad::vm::llvm::compile(rev, {code, code_size});

// 캐시에 저장
cached_llvm_code_[rev].insert({code_hash, ptr});
```

```
return ptr;
}

evmc::Result VM::execute_llvm(
evmc_revision rev,
evmc::bytes32 const  &code_hash,
evmc_host_interface const  *host,
evmc_host_context *context,
evmc_message const  *msg,
uint8_t  const  *code,
size_t  code_size)
{
auto ctx = runtime::Context::from(
memory_allocator_, host, context, msg, {code, code_size});

auto  const stack_ptr = stack_allocator_.allocate();
uint256_t *evm_stack = reinterpret_cast<uint256_t *>(stack_ptr.get());

// LLVM 코드 가져오기 및 실행
auto llvm = cache_llvm(rev, code_hash, code, code_size);
monad::vm::llvm::execute(*llvm, ctx, evm_stack);
```

```
return  ctx.copy_to_evmc_result();

}
```

5.3 LLVM vs asmjit 비교

| 특성 | asmjit | LLVM |

|------|--------|------|

| 컴파일 속도 | 매우 빠름 (~1ms) | 느림 (~10-100ms) |

| 최적화 수준 | 기본 최적화 | 고급 최적화 |

| 코드 크기 | 작음 | 더 작음 |

| 실행 성능 | 우수 | 최고 |

| 메모리 사용량 | 적음 | 많음 |

| 사용 사례 | 기본 JIT | 오프라인 AOT |

Monad의 선택:

- 기본적으로 asmjit 사용 (빠른 컴파일)
- LLVM은 선택적 기능 (고급 최적화 필요 시)

6. 성능 최적화 기법 요약

6.1 인터프리터 최적화

1. **Tail Call 최적화**: 스택 오버플로우 방지
2. **직접 스레드 디스패치**: 함수 포인터 테이블로 빠른 명령어 전환
3. **컴파일 타임 검증**: 불필요한 런타임 검증 제거
4. **인라인 함수**: 작은 함수를 인라인으로 전개

6.2 컴파일러 최적화

1. **비동기 컴파일**: 백그라운드 스레드로 실행 지연 최소화
2. **가스 기반 트리거**: 자주 실행되는 코드만 컴파일
3. **가중치 기반 캐싱**: 효율적인 메모리 사용
4. **Dual Mapping**: W^X 보안 정책 준수하면서 성능 유지

6.3 메모리 관리 최적화

1. **스택/메모리 재사용**: 할당자 캐시로 할당 오버헤드 감소
2. **사전 할당**: 필요한 메모리를 미리 할당
3. **RAII 패턴**: 자동 리소스 관리

6.4 동시성 최적화

1. **락 프리 자료구조**: TBB concurrent 컨테이너 사용
2. **원자적 연산**: 최소한의 동기화
3. **조건 변수**: 효율적인 스레드 대기

7. 코드 실행 흐름 다이어그램

```
┌─────────────────────────────────────────────────────────────┐
│ VM::execute() │
│ (실행 진입점) │
└─────────────────────────────────────────────────────────────┘
│
▼
```

```
┌──────────────────────────────────────────────────────────────┐
| Varcode 확인 |
| - Intercode (중간 표현) |
| - Nativecode (컴파일된 코드, 옵션) |
└──────────────────────────────────────────────────────────────┘
|
┌──────────┴──────────┐
| |
Nativecode 존재? Nativecode 없음
| |
▼ ▼
┌──────────────────┐ ┌──────────────────┐
| 버전 확인 | | 캐시 Warm? |
└──────────────────┘ └──────────────────┘
| |
┌──────┴──────┐ ┌──────┴──────┐
| | | |
일치함 불일치 Yes No
| | | |
▼ ▼ ▼ ▼
┌────────┐ ┌──────────────┐ ┌──────────────┐
|네이티브| |인터프리터 | |즉시 컴파일 |
|코드실행| |+ 재컴파일 | |+ 인터프리터 |
```

```
└────────┘ └──────────────┘ └──────────────┘

|

▼

┌──────────────────┐

|가스 임계값 확인 |

└──────────────────┘

|

초과 시 컴파일

▼

┌────────────────────────┐

| 백그라운드 컴파일 |

| (compile_loop) |

└────────────────────────┘

|

▼

┌────────────────────────┐

| Varcode 캐시 저장 |

└────────────────────────┘
```

8. 주요 데이터 구조

8.1 Varcode

```
struct Varcode {
SharedIntercode intercode; // 중간 표현 (항상 존재)
SharedNativecode nativecode; // 네이티브 코드 (옵션)
std::atomic<uint64_t> gas_used; // 누적 가스 사용량
};
```

8.2 Intercode

```
class Intercode {
std::uint8_t  const *padded_code_; // 패딩된 바이트코드
code_size_t code_size_; // 원본 코드 크기
JumpdestMap jumpdest_map_; // JUMPDEST 비트맵
};
```

8.3 Nativecode

```
class Nativecode {
asmjit::JitRuntime &rt_; // JIT 런타임
uint64_t chain_id_; // EVM 버전 ID
void *code_; // 실행 가능 코드
entrypoint_t entrypoint_; // 엔트리포인트 함수
native_code_size_t code_size_; // 코드 크기
ErrorCode error_code_; // 에러 코드
```

```
};
```

8.4 CompilerConfig

```
struct CompilerConfig {
native_code_size_t max_code_size_offset; // 코드 크기 오프셋
// 추가 컴파일러 설정...
};
```

9. 성능 통계 및 모니터링

9.1 컴파일러 통계

```
struct CompilerStats
{
utils::EuclidMean<uint64_t> avg_native_code_size_;
utils::EuclidMean<uint64_t> avg_compiled_bytecode_size_;
utils::GeoMean<double> avg_native_code_ratio_;
utils::EuclidMean<int64_t> avg_compile_time_;
std::atomic<uint32_t> max_native_code_size_{0};
std::atomic<uint32_t> max_compiled_bytecode_size_{0};
std::atomic<uint64_t> num_compiled_contracts_{0};
std::atomic<int64_t> max_compile_time_{0};
```

```
std::atomic<uint64_t> num_unexpected_compilation_errors_{0};
std::atomic<uint64_t> num_size_out_of_bound_compilation_errors_{0};
};
```

수집되는 메트릭:

- 평균/최대 네이티브 코드 크기
- 평균/최대 바이트코드 크기
- 네이티브 코드 비율 (네이티브 크기 / 바이트코드 크기)
- 평균/최대 컴파일 시간
- 컴파일된 컨트랙트 수
- 컴파일 에러 수

9.2 VM 통계

```
struct VMStats
{
std::atomic<uint64_t> num_execute_intercode_{0};
std::atomic<uint64_t> num_execute_native_{0};
std::atomic<uint64_t> num_execute_bytecode_{0};
};
```

실행 모드 추적:

- 인터프리터 실행 횟수
- 네이티브 코드 실행 횟수

- 직접 바이트코드 실행 횟수

10. 실제 사용 예시

10.1 VM 초기화

```
// VM 생성 (비동기 컴파일 활성화)
VM vm(
true, // enable_async
1024 * 1024, // max_stack_cache (1MB)
16 * 1024 * 1024 // max_memory_cache (16MB)
);
```

10.2 코드 실행

```
// 1. Intercode 생성 또는 캐시 조회
auto code_hash = keccak256(bytecode);
auto vcode = compiler.try_insert_varcode(code_hash, icode);

// 2. 실행
auto result = vm.execute<traits>(
host, // Host 인터페이스
&msg, // 메시지
code_hash, // 코드 해시
```

```
vcode // Varcode
);


// 3. 결과 확인
if (result.status_code == EVMC_SUCCESS) {
// 성공
} else {
// 실패 처리
}
```

10.3 컴파일 통계 출력

```
std::string stats = compiler.print_stats();
// 출력 예시:
// avg_native_code_size=2048B,avg_compiled_bytecode_size=512B
// avg_native_code_ratio=4.00,max_native_code_size=8192B
// num_compiled_contracts=1000,avg_compile_time=850μs
// varcode_cache_size=1000,varcode_cache_weight=4096kB
```

11. 최적화 효과 분석

11.1 인터프리터 vs 네이티브 코드

성능 비교 (추정):

- 인터프리터: 기준 성능 (1x)
- asmjit 네이티브 코드: 5-10x 빠름
- LLVM 네이티브 코드: 10-20x 빠름

트레이드오프:

- 인터프리터: 즉시 실행, 낮은 성능
- asmjit: 빠른 컴파일 (~1ms), 우수한 성능
- LLVM: 느린 컴파일 (~10-100ms), 최고 성능

11.2 비동기 컴파일 효과

지연 시간 비교:

- 동기 컴파일: 첫 실행 시 1-100ms 지연
- 비동기 컴파일: 첫 실행 지연 없음, 백그라운드에서 컴파일

메모리 사용량:

- 작업 큐 소프트 리밋: 1000개 (기본값)
- 평균 Intercode 크기: ~10KB
- 최대 메모리 오버헤드: ~10MB

11.3 캐싱 효과

캐시 히트율:

- Warm 캐시 (75% 이상): 높은 히트율 (~90%)
- Cold 캐시: 낮은 히트율 (~10%)

메모리 효율:

- 가중치 기반 LRU: 작은 컨트랙트 우선 유지
- 최대 캐시 크기: 4GB (기본값)

- 실제 사용량: 동적 조정

12. 참고 자료

12.1 관련 라이브러리

asmjit:

- GitHub: https://github.com/asmjit/asmjit
- 문서: https://asmjit.com/doc/
- 용도: 런타임 x86/x64 어셈블리 코드 생성

Intel TBB:

- GitHub: https://github.com/oneapi-src/oneTBB
- 문서: https://oneapi-src.github.io/oneTBB/
- 용도: 동시성 자료구조 및 병렬 알고리즘

EVMC:

- GitHub: https://github.com/ethereum/evmc
- 문서: https://evmc.ethereum.org/
- 용도: Ethereum VM C API

LLVM:

- 웹사이트: https://llvm.org/
- 문서: https://llvm.org/docs/
- 용도: 고급 최적화 컴파일러 백엔드

12.2 EVM 명세

- Ethereum Yellow Paper: https://ethereum.github.io/yellowpaper/paper.pdf

- EVM Opcodes: https://www.evm.codes/
- EIP (Ethereum Improvement Proposals): https://eips.ethereum.org/

12.3 JIT 컴파일 기법

관련 논문:

1. "Efficient Interpretation using Quickening" (2015)
2. "Trace-based Just-in-Time Type Specialization for Dynamic Languages" (2009)
3. "A Brief History of Just-In-Time" (2003)

최적화 기법:

- Tail Call Optimization
- Direct Threading
- Inline Caching
- Type Specialization
- Constant Folding

12.4 Monad 특화 최적화

핵심 차별점:

1. **하이브리드 실행**: 인터프리터와 JIT의 동적 전환
2. **가스 기반 트리거**: 실행 빈도에 따른 적응형 컴파일
3. **비동기 컴파일**: 실행 지연 최소화
4. **가중치 기반 캐싱**: 효율적인 메모리 관리

13. 결론

Monad의 VM 네이티브 컴파일 메커니즘은 다음과 같은 핵심 기법을 통해 극한의 성능을 달성합니다:

1. **하이브리드 실행 모델**: 인터프리터의 빠른 시작과 JIT의 높은 성능을 결합
2. **비동기 컴파일**: 백그라운드 스레드로 실행 지연 최소화
3. **적응형 컴파일**: 가스 사용량 기반으로 컴파일 여부 결정
4. **효율적인 캐싱**: 가중치 기반 LRU로 메모리 효율 극대화
5. **고성능 코드 생성**: asmjit과 LLVM을 통한 최적화된 네이티브 코드

이러한 최적화 기법들은 Monad가 초고속 블록체인을 구현하는 핵심 기반이 됩니다.